

CSE420

Compiler Design

Introduction to Syntax-Directed Translation
& Construction of the Parse Tree Using SDD Rules

Prepared by Azmari Sultana

Table of Contents

1. Introduction to Syntax-Directed Translation	3
1.1 What is a Syntax-Directed Definition (SDD)?	3
1.2 Inherited and Synthesized Attributes	3
1.3 S-Attributed and L-Attributed SDDs	5
1.4 Dependency Graphs and Evaluation Order	6
1.5 Syntax-Directed Translation Schemes (SDT)	9
2. Construction of the Parse Tree Using SDD Rules	14
2.1 Parse Tree for $(id + id * id / id)$	14
2.2 Parse Tree for $(id / id + id * id)$	16
3. AST Construction with a Reduce-Count Attribute	18
3.1 General Rule	18
3.2 Reduce Count for $(p+q)*(r/s+t)$	19
3.3 Reduce Count for $(a*b+c)/(d+e)$	21
4. SDT-1 vs SDT-2 - Placement of Semantic Actions	24
5. Filling In Missing SDT Rules for Syntax-Tree Construction	24
5.1 Input: $x + ((y+z)*w)$	25
5.2 Input: $x + ((y*z)+w)$	27
6. Practice Sheet	29
6.1 Parse Tree of Nodes	29
6.2 Reduce-Count SDD	31
6.3 Filling In Missing SDT Rules	33
6.4 Filling In Missing SDT Rules - Field-Based Style	35

1. Introduction to Syntax-Directed Translation

1.1 What is a Syntax-Directed Definition (SDD)?

A Syntax-Directed Definition (SDD) is a context-free grammar together with attributes and semantic rules attached to it.

- Attributes are attached to grammar symbols (e.g. $E.val$, $T.node$).
- Semantic rules are attached to productions and describe how to compute an attribute's value.
- An SDD with no side effects is called an attribute grammar.
- SDDs are more readable and used for specification; SDTs (Section 1.5) embed the same rules as actions and are more efficient for implementation.

1.2 Inherited and Synthesized Attributes

Every grammar symbol in an SDD can carry attributes (values such as a type, a number, or a tree pointer). Depending on which direction the information flows in the parse tree, an attribute is classified as either synthesized or inherited.

Synthesized attribute — flows UP (children $\rightarrow$ parent)

A synthesized attribute of a nonterminal A at a parse-tree node N is computed using only attribute values from the CHILDREN of N (and, optionally, N 's own inherited attributes). The rule for it is written on the production that has A as its HEAD.

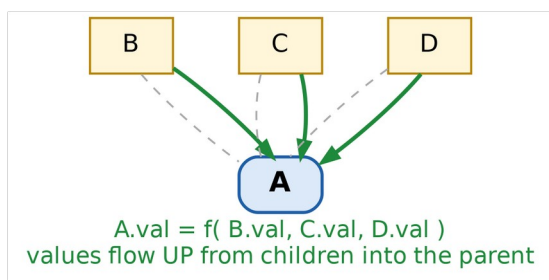
Example: for $E \rightarrow E1 + T$, the rule $E.val = E1.val + T.val$ computes $E.val$ (at the parent node E) purely from the val attributes of its two children, $E1$ and T . Information travels bottom-up, from children to parent.

Inherited attribute — flows DOWN / SIDEWAYS (parent or left sibling $\rightarrow$ child)

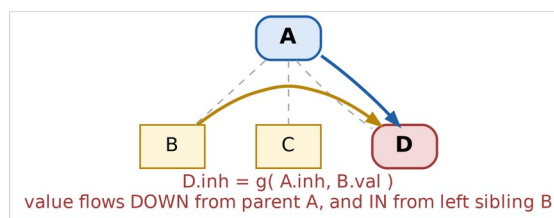
An inherited attribute of a nonterminal B at a parse-tree node N is computed using attribute values from N 's PARENT and/or N 's SIBLINGS. The rule for it is written on the production where B appears in the BODY (i.e. the rule sits at the parent's production, not B 's own production).

Example: for $T \rightarrow F T'$, the rule $T'.inh = F.val$ computes $T'.inh$ (at the child node T') using $F.val$, the value of T' 's left sibling F . Information travels top-down or left-to-right, from parent/sibling into a child.

Quick visual



Synthesized: flows bottom-up (children $\rightarrow$ parent)



Inherited: flows top-down / sideways (parent, sibling $\rightarrow$ child)

Synthesized vs Inherited — Comparison Table

Aspect	Synthesized Attribute	Inherited Attribute
Direction of flow	Bottom-up (children $\rightarrow$ parent)	Top-down / left-to-right (parent or left sibling $\rightarrow$ child)
Computed from	Attribute values of its own children (and its own inherited attributes)	Attribute values of its parent and/or its left siblings
Rule attached to	The production where this nonterminal is	The production where this nonterminal

	the HEAD	appears in the BODY
Can terminals have it?	Yes — lexical values like lexval (supplied by the lexical analyzer)	No — terminals never have inherited attributes
Natural evaluation order	Postorder traversal / bottom-up (LR) parsing	Preorder-ish, left-to-right / top-down (LL) parsing
Used alone gives which SDD class?	S-attributed SDD (all attributes synthesized)	Needed (together with synthesized) for L-attributed SDD
Example rule	$E.val = E1.val + T.val$ (at head E)	$T'.inh = F.val$ (at child T' , from left sibling F)

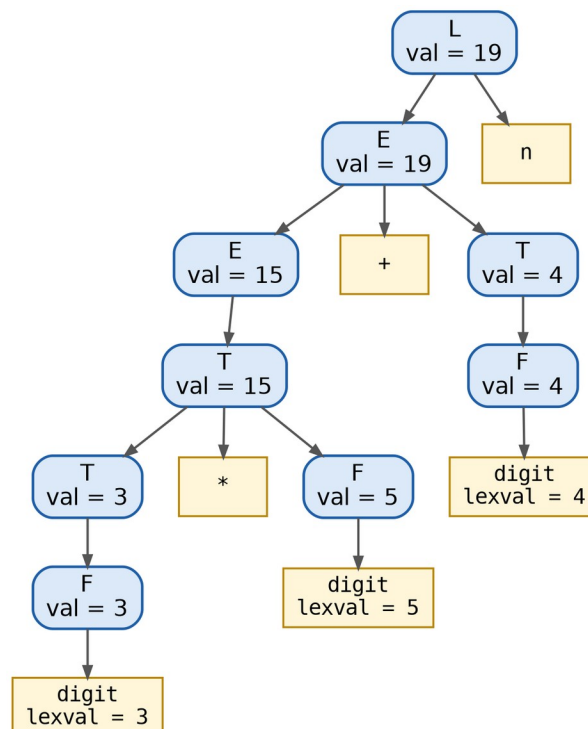
- Terminals can only have synthesized attributes — their lexical values (such as lexval) are supplied directly by the lexical analyzer.
- A synthesized attribute at a node may use the inherited attribute values of that same node.

Example 5.1 - Desk Calculator SDD (S-attributed)

Production	Semantic Rule
$L \rightarrow E n$	$L.val = E.val$
$E \rightarrow E1 + T$	$E.val = E1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T1 * F$	$T.val = T1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

For input $3 * 5 + 4 n$, the annotated parse tree gives $T.val = 15$ (from $3 * 5$) and $E.val = 19$ ($15 + 4$), so $L.val = 19$.

Annotated parse tree for $3 * 5 + 4 n$:

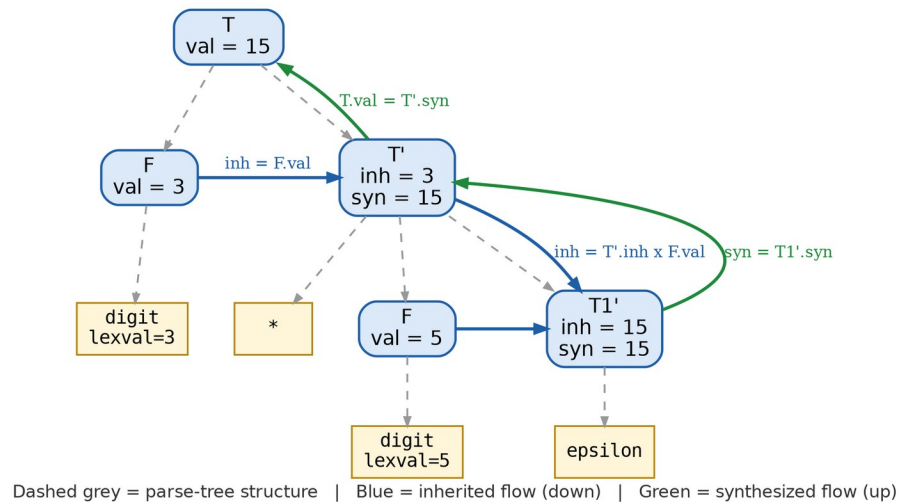


Example 5.3 - Why Inherited Attributes are Needed

With a top-down (non-left-recursive) grammar $T \rightarrow F T' , T' \rightarrow * F T1' \mid \epsilon$, the operand F and the operator $*$ end up in different subtrees. An inherited attribute $T'.inh$ passes the left operand down to where the operator sits, and a synthesized attribute $T'.syn$ passes the result back up.

Production	Semantic Rule
$T \rightarrow F T'$	$T'.inh = F.val ; T.val = T'.syn$
$T' \rightarrow * F T1'$	$T1'.inh = T'.inh \times F.val ; T'.syn = T1'.syn$
$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Attribute-flow diagram for input 3 * 5:



1.3 S-Attributed and L-Attributed SDDs

S-attributed SDD

Every attribute is synthesized. Attributes can be evaluated in any bottom-up order (e.g. a postorder traversal). This class fits naturally with bottom-up (LR) parsers, since each reduce step corresponds to a postorder visit.

L-attributed SDD

Every attribute is either (a) synthesized, or (b) inherited, where an inherited attribute of a symbol X_i in a production $A \rightarrow X_1 X_2 \dots X_n$ may depend only on:

- Inherited attributes of the head A , and/or
- Inherited or synthesized attributes of symbols strictly to the LEFT of X_i ($X_1 \dots X_{i-1}$), and/or
- Inherited or synthesized attributes of X_i itself (as long as there is no cycle).

In short: information may only flow left-to-right and top-down, never right-to-left. Every S-attributed SDD is automatically L-attributed. L-attributed SDDs can be evaluated during a top-down parse (or during a bottom-up parse of an LL grammar).

Theory Questions

Q: What is the difference between inherited and synthesized attributes?

Ans: A synthesized attribute of a node is computed from its own children's attributes (bottom-up flow); an inherited attribute of a node is computed from its parent's and/or siblings' attributes (top-down / left-to-right flow).

Q: What is an S-attributed SDD? What is an L-attributed SDD?

Ans: S-attributed: all attributes are synthesized; evaluated bottom-up, suited to LR parsers. L-attributed: attributes are synthesized, or inherited only from the head or from symbols to the left (no right-to-left).

dependency); suited to top-down (LL) parsers. Every S-attributed SDD is L-attributed, but not every L-attributed SDD is S-attributed.

Q: Why can't terminals have inherited attributes?

Ans: A terminal's attribute value (e.g. `lexval`) is supplied directly by the lexical analyzer, not computed by any semantic rule inside the SDD, so the inherited/synthesized distinction — which applies to rule-computed attributes — does not apply to terminals.

1.4 Dependency Graphs and Evaluation Order

A dependency graph shows, for one specific parse tree, which attribute instances depend on which others. An edge from attribute M to attribute N means "M must be computed before N."

- Synthesized attribute A.b computed from X.c → edge from X.c to A.b.
- Inherited attribute B.c computed from X.a → edge from X.a to B.c.

The only valid evaluation orders are topological sorts of the dependency graph — an order $N_1, N_2, \dots, N_k$ such that every edge points from a lower index to a higher index.

- If the dependency graph has a cycle, no evaluation order exists, so the SDD cannot be evaluated on that parse tree.
- If there is no cycle, at least one topological sort always exists.

Deciding, in general, whether an SDD's dependency graph can ever have a cycle is computationally hard. This is exactly why we restrict ourselves to the safe subclasses — S-attributed and L-attributed SDDs — which are guaranteed to be acyclic and can be evaluated efficiently alongside parsing, without ever building the dependency graph explicitly.

Semantic Rules with Controlled Side Effects

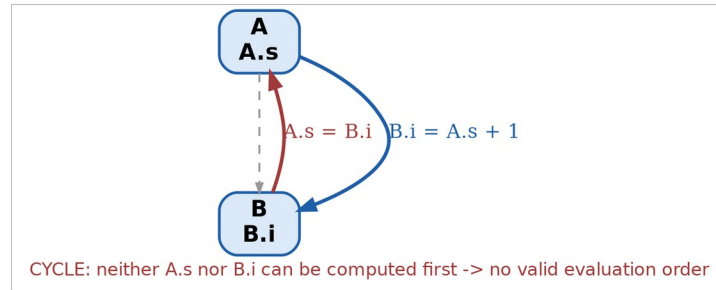
In practice, translations involve side effects — a desk calculator might print a result, or a code generator might enter a type into a symbol table. Since SDDs otherwise allow any evaluation order consistent with the dependency graph, side effects must be controlled in one of two ways:

- Permit incidental side effects that do not constrain attribute evaluation — i.e. allow the side effect to happen under ANY topological sort, as long as the result is still "correct". Example: changing $L.val = E.val$ to a rule `print(E.val)` at production $L \rightarrow E n$. This is treated as a dummy synthesized attribute of L, and since `print` happens only after `E.val` is already computed, any evaluation order gives the same result.
- Constrain the allowable evaluation orders so that the same translation is produced for any allowable order — these constraints act like extra implicit edges added to the dependency graph. Example: the `addType(id.entry, L.inh)` calls in Example 5.10 (Section 1.4 below) — each call only depends on that specific identifier's own entry and the inherited type, so entries can be updated in any order without affecting correctness.

Worked Examples (from the reference textbook)

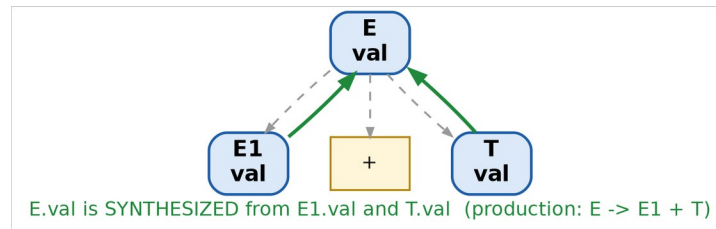
Example 5.2 - A circular dependency graph (Fig. 5.2)

Consider $A \rightarrow B$ with rules $A.s = B.i$ and $B.i = A.s + 1$. These rules are circular: A.s cannot be computed without first knowing B.i, and B.i cannot be computed without first knowing A.s. Since neither can go first, there is no topological sort — this SDD simply cannot be evaluated on this parse tree.



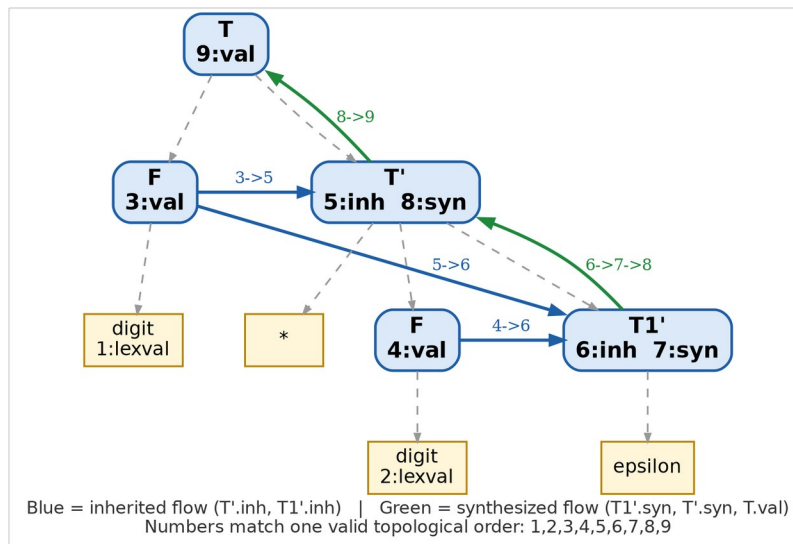
Example 5.4 - Dependency graph for $E \rightarrow E1 + T$ (Fig. 5.6)

At every node N labeled E built from this production, the synthesized attribute val at N is computed from val at its two children E1 and T. The dashed edges below are the parse-tree edges; the solid green edges are the dependency-graph edges (attribute-value flow).



Example 5.5 - A complete dependency graph for $\text{digit} * \text{digit}$ (Fig. 5.7)

Using the L-attributed grammar $T \rightarrow F T'$, $T' \rightarrow * F T1'$, $T1' \rightarrow \epsilon$ from Example 5.3, here is the full dependency graph for input $3 * 5$, with nodes numbered exactly as in the textbook. Node 1 and 2 are the `digit.lexval` leaves; 3 and 4 are `F.val`; 5 and 6 are the inherited `T'.inh` / `T1'.inh`; 7 and 8 are the synthesized `T1'.syn` / `T'.syn`; 9 is the final `T.val`.



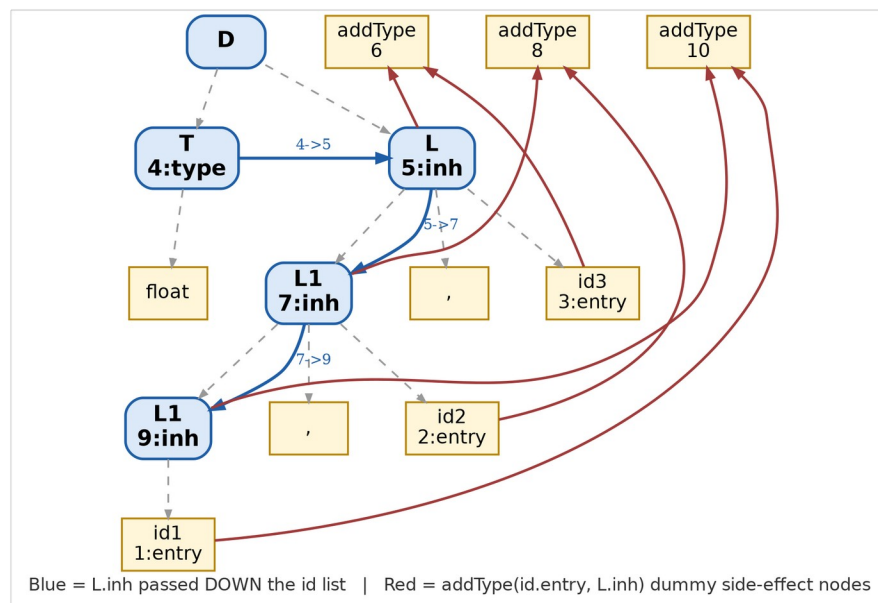
One valid topological sort of this graph is simply 1, 2, 3, 4, 5, 6, 7, 8, 9 — every edge in the graph goes from a lower-numbered node to a higher-numbered one, so evaluating in this order is always safe.

Example 5.10 - SDD and dependency graph for type declarations (Fig. 5.8, Fig. 5.9)

This SDD enters a declared type into the symbol table for every identifier in a list, using an inherited attribute L.inh to pass the type down the list of identifiers.

Production	Semantic Rules
$D \rightarrow T L$	$L.inh = T.type$
$T \rightarrow \text{int}$	$T.type = \text{integer}$
$T \rightarrow \text{float}$	$T.type = \text{float}$
$L \rightarrow L1, id$	$L1.inh = L.inh ; \text{addType}(id.entry, L.inh)$
$L \rightarrow id$	$\text{addType}(id.entry, L.inh)$

For the declaration `float id1, id2, id3`, the dependency graph below shows $T.type$ (node 4) flowing down through each $L.inh$ (nodes 5, 7, 9), and each `addType` call (the dummy synthesized nodes 6, 8, 10) firing once the type and that identifier's entry are both available.



Semantic Rules with Controlled Side Effects

In practice, translations need side effects — printing a result, or entering a type into a symbol table (exactly like `addType` above). SDDs allow LIMITED side effects, controlled in one of two ways:

- Permit incidental side effects that do NOT constrain evaluation order — the side effect is safe under ANY topological sort of the dependency graph. Example: changing $L.val = E.val$ (which saves the result) to `print(E.val)` at the end of $L \rightarrow E n$. Since `print` happens after $E.val$ is already computed, it gives the same result regardless of which valid order was used.
- Constrain the allowable evaluation orders so that every allowed order gives the same translation — these constraints act like extra implicit edges added to the dependency graph.

Theory Questions

Q: What is a dependency graph and what is it used for?

Ans: A dependency graph depicts the flow of information between attribute instances at the nodes of a specific parse tree. It is used to find a valid order (a topological sort) in which the attributes can be evaluated.

Q: What does it mean if a dependency graph has a cycle?

Ans: It means there is no valid evaluation order — some attribute's value depends, directly or indirectly, on itself — so the SDD cannot be evaluated on that parse tree.

Q: Why do we prefer S-attributed / L-attributed SDDs in practice?

Ans: Because checking whether a general SDD's dependency graph is always cycle-free is computationally very expensive. S-attributed and L-attributed SDDs are, by construction, guaranteed to be acyclic, so they can be evaluated efficiently during a single bottom-up or top-down parse without needing to build and analyze a dependency graph.

1.5 Syntax-Directed Translation Schemes (SDT)

In plain words:

An SDT is just a grammar with little pieces of code — called actions — dropped in between the symbols of a production. Think of a recipe: normally you list ALL the ingredients first and then the steps. An SDT is more like a recipe where a small instruction ("stir now", "print the total") is written right in the middle of the ingredient list, at the exact spot where it should happen while you are reading through it.

Formally: an SDT is a context-free grammar with program fragments, called semantic actions, embedded inside production bodies (written inside { } braces). An action fires (runs) as soon as everything to its LEFT in that production has already been matched.

- Bottom-up parse: an action fires the moment its position on the parsing stack is reached — i.e. as soon as everything before it has been shifted/reduced.
- Top-down parse: an action fires just before the parser is about to expand or match whatever comes right after it.

An SDT's actions can be represented as marker nonterminals ($M \rightarrow \epsilon$) placed exactly where the action sits. If the grammar with these markers can still be parsed by the chosen method, the SDT can be executed during parsing.

1.5.1 Postfix SDT (S-attributed, actions at the end)

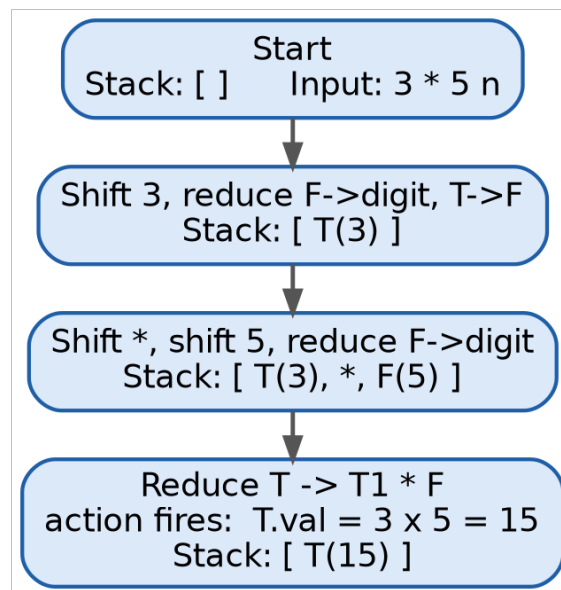
In plain words:

This is the EASY case. If every action is written at the very end of a production, the action only ever needs values that are already fully known (the values of everything that came before it). So we can always evaluate it the moment that production is recognised — no waiting, no guessing what comes next.

If the SDD is S-attributed and the grammar is LR (bottom-up parsable), every action can be placed at the very end of its production body. This is called a postfix SDT — the simplest kind, since actions fire exactly when a reduction happens.

Production	Action
$L \rightarrow E n$	{ print(E.val); }
$E \rightarrow E1 + T$	{ E.val = E1.val + T.val; }
$E \rightarrow T$	{ E.val = T.val; }
$T \rightarrow T1 * F$	{ T.val = T1.val $\times$ F.val; }
$T \rightarrow F$	{ T.val = F.val; }
$F \rightarrow (E)$	{ F.val = E.val; }
$F \rightarrow \text{digit}$	{ F.val = digit.lexval; }

Watch it happen - parsing $3 * 5$ step by step:



Parser-stack implementation:

Attributes are stored alongside grammar symbols on the LR parsing stack (a `val` field in every stack record). When reducing $A \rightarrow XYZ$, the attributes of X, Y, Z sit at known stack offsets (top, top-1, top-2); the action computes A's attribute and places it where X used to be, then pops the rest.

1.5.2 SDTs with Actions Inside Productions

In plain words:

This is the HARD case. If an action sits at the START or MIDDLE of a production, it may need to run before the parser has even seen the rest of that production. That is like being asked to shout out "it's a plus sign!" before you've actually read whether the sentence contains a plus sign at all — you simply cannot know yet.

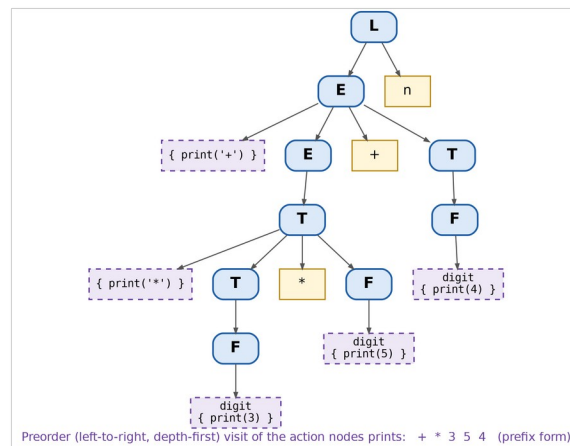
An action can appear anywhere in a production body, not only at the end. Not every such SDT can be executed during parsing — some require information (such as whether a $+$ or $*$ will appear) before the parser can possibly know it.

Example 5.16 - A problematic SDT (cannot be done during parsing)

- 1) $L \rightarrow E n$
- 2) $E \rightarrow \{ \text{print}('+'); \} E_1 + T$
- 3) $E \rightarrow T$
- 4) $T \rightarrow \{ \text{print}('*'); \} T_1 * F$
- 5) $T \rightarrow F$
- 6) $F \rightarrow (E)$
- 7) $F \rightarrow \text{digit} \{ \text{print}(\text{digit.lexval}); \}$

Here the action `print('+')` sits at the START of the production, but the parser cannot know at that point whether a $+$ will actually appear later — it would need unbounded look-ahead. This SDT cannot be implemented during top-down or bottom-up parsing; it can only be done by first building the full parse tree, then performing a preorder traversal and firing actions as each node is visited (this always works for ANY SDT, but is less efficient since the tree must be built explicitly).

Figure 5.22 - Parse tree with actions embedded, for input 3 * 5 + 4:



Visiting the tree above left-to-right, top-to-bottom (preorder) and firing each action as you reach it prints: + * 3 5 4 — the prefix form of 3*5+4. This only works because we waited until the WHOLE tree was built first.

1.5.3 Eliminating Left Recursion from SDTs

In plain words:

A top-down parser reads left to right and cannot handle a rule like $A \rightarrow A \alpha$, because to expand A it would immediately need to expand A again, forever, before reading any real input. The fix is the same trick used for plain grammars (Section 4.3.3): move the repetition into a new helper nonterminal R that eats one α at a time and carries the running result along with it, instead of building it top-down.

Left-recursive grammars cannot be parsed top-down, so left recursion must first be removed. The standard transformation:

$A \rightarrow A \alpha \mid \beta$ becomes $A \rightarrow \beta R$
 $R \rightarrow \alpha R \mid \epsilon$

Case 1 - actions only print (order matters, not attribute values):

Simply treat the actions as if they were terminal symbols and carry them through the same transformation — the order in which they print is preserved either way.

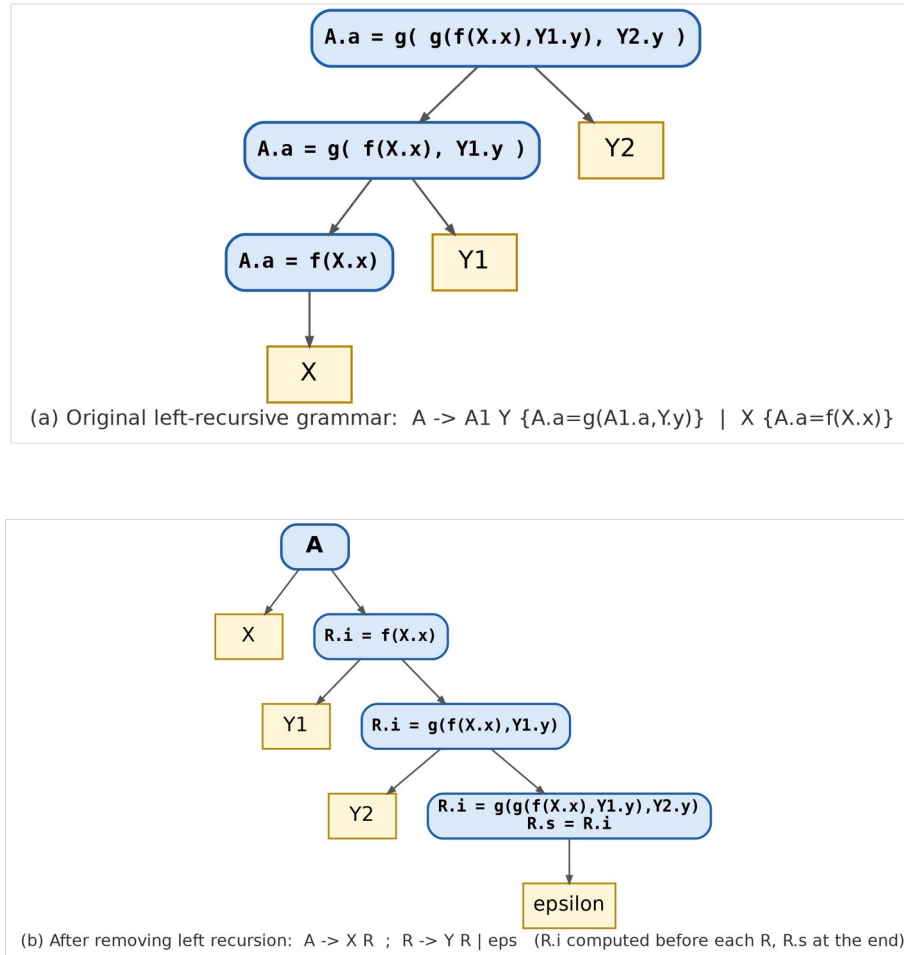
Case 2 - actions compute attributes (S-attributed SDD):

Given the schema $A \rightarrow A1 Y \{A.a = g(A1.a, Y.y)\}$ and $A \rightarrow X \{A.a = f(X.x)\}$ — introduce an inherited attribute $R.i$ that carries forward the accumulating result, and a synthesized attribute $R.s$ that sends the final value back up:

$A \rightarrow X \{ R.i = f(X.x) \} \quad R \{ A.a = R.s \}$
 $R \rightarrow Y \{ R1.i = g(R.i, Y.y) \} \quad R1 \{ R.s = R1.s \}$
 $R \rightarrow \epsilon \{ R.s = R.i \}$

The inherited attribute $R.i$ is evaluated immediately BEFORE each use of R ; the synthesized attributes $A.a$ and $R.s$ are evaluated at the END of their productions — this left-to-right placement is exactly what makes the transformation work.

Figure 5.23 - Eliminating left recursion from a postfix SDT:



Compare the two trees: (a) is how the ORIGINAL left-recursive grammar would build up the value — nested calls to g , growing to the left, which a top-down parser cannot construct. (b) is the SAME final value, but built by R passing the running total DOWN as an inherited attribute ($R.i$) each time it eats one more Y , until it hits ϵ and sends the finished value back UP as $R.s$.

1.5.4 SDTs for L-Attributed Definitions

In plain words:

This is the general recipe for turning ANY L-attributed SDD into something a top-down parser can execute directly, with no tree-building needed. There are only two placement rules to remember.

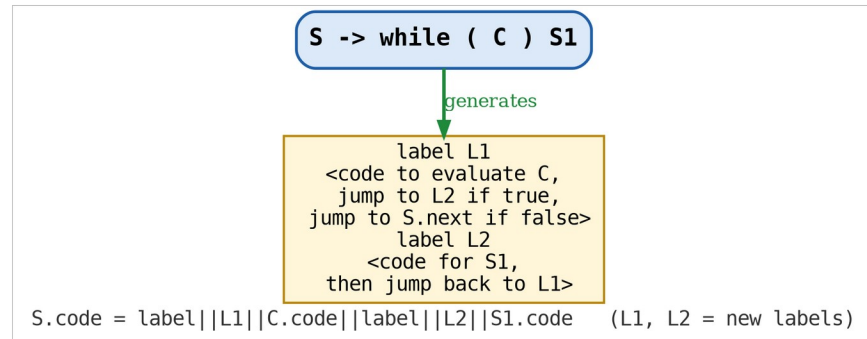
For a general L-attributed SDD (grammar parsable top-down), convert it into an SDT using two rules:

- Rule 1: Place the action that computes the INHERITED attributes of a nonterminal immediately BEFORE that occurrence of the nonterminal in the production body.
- Rule 2: Place the action that computes a SYNTHESIZED attribute of the head at the END of the production body.

A concrete example: generating code for a while-loop

Consider $S \rightarrow \text{while} (C) S1$ — a real compiler needs to generate jump/label code for this. $S.\text{next}$ tells S where to jump to when it's done; $C.\text{true} / C.\text{false}$ tell the condition where to jump for each outcome; $S.\text{code}$ and $C.\text{code}$ are the actual generated instructions.

Attribute	Meaning
$S.\text{next}$ (inherited)	label of the code to run right after this while-loop finishes
$S.\text{code}$ (synthesized)	the generated instructions for this statement
$C.\text{true} / C.\text{false}$ (inherited)	labels to jump to if the condition is true / false
$C.\text{code}$ (synthesized)	the generated instructions for evaluating the condition



Applying Rule 1 and Rule 2 above gives this SDT — notice the inherited attributes ($C.\text{false}$, $C.\text{true}$, $S1.\text{next}$) are all computed BEFORE the nonterminal they belong to, and the synthesized $S.\text{code}$ is computed at the very END:

```
S → while (
    { L1 = new(); L2 = new(); C.false = S.next; C.true = L2; }
  C )
    { S1.next = L1; }
  S1
    { S.code = label||L1||C.code||label||L2||S1.code; }
```

Another real example: typesetting (Eqn/TeX-style subscripts)

SDTs aren't just for programming languages. Eqn (an ancestor of TeX's math mode) typesets expressions like a sub i sub j to print a_{ij} — a subscript box shrinks by 30%, drops slightly, and sits to the right of its base box. The grammar $B \rightarrow B1 B2 \mid B1 \text{ sub } B2 \mid (B1) \mid \text{text}$ uses an inherited attribute $B.\text{ps}$ (point size, passed DOWN so subscripts know how much to shrink) and synthesized attributes $B.\text{ht} / B.\text{dp}$ (height/depth, computed UP once the sub-boxes are sized) — exactly the same L-attributed pattern as the while-loop example above, just applied to typesetting instead of code generation.

Theory Questions

Q: What is the difference between an SDD and an SDT?

Ans: An SDD attaches semantic RULES to productions and allows any evaluation order consistent with the dependency graph (a specification, with no fixed order). An SDT embeds semantic ACTIONS at specific positions inside the production body and executes them in strict left-to-right (preorder) order — it is closer to an implementation.

Q: What is a postfix SDT and when can it be used?

Ans: A postfix SDT places every action at the end of its production body. It can be used exactly when the underlying grammar is LR (bottom-up parsable) and the SDD is S-attributed — the action then fires naturally at the moment of reduction.

Q: What is a marker nonterminal and why is it useful?

Ans: A marker nonterminal M is a dummy nonterminal with a single production $M \rightarrow \epsilon$, inserted at the exact position of an embedded action. If the grammar with these markers inserted can still be parsed by the chosen method without new conflicts, the SDT can be executed during that parse; if the markers create conflicts, the SDT cannot be implemented during parsing.

Q: Give one example of an SDT that cannot be executed during parsing, and explain why.

Ans: The infix-to-prefix SDT that prints '+' or '*' at the START of $E \rightarrow \{\text{print}('+')\} E1 + T$ and $T \rightarrow \{\text{print}('*')\} T1 * F$ cannot be executed during parsing, because the parser would have to decide to print '+' or '*' before it has seen whether that operator actually appears later in the input — this needs unbounded look-ahead, which parsers do not have.

Q: How do you preserve correct attribute computation while eliminating left recursion from an S-attributed SDT?

Ans: Introduce a new nonterminal R for the "remainder", give it an inherited attribute $R.i$ that carries the accumulated result forward (initialised from A 's first operand) and a synthesized attribute $R.s$ that carries the final result back up. $R.i$ is computed immediately before each recursive use of R , and $R.s / A.a$ are computed at the end of their productions, following the L-attributed placement rules.

2. Construction of the Parse Tree Using SDD Rules

This is the SIMULATION portion of the syllabus: given an attribute grammar whose rules build an explicit tree of Node/Leaf objects (instead of just a value), draw the resulting tree for a given input string, respecting the exact order in which children are added by each rule.

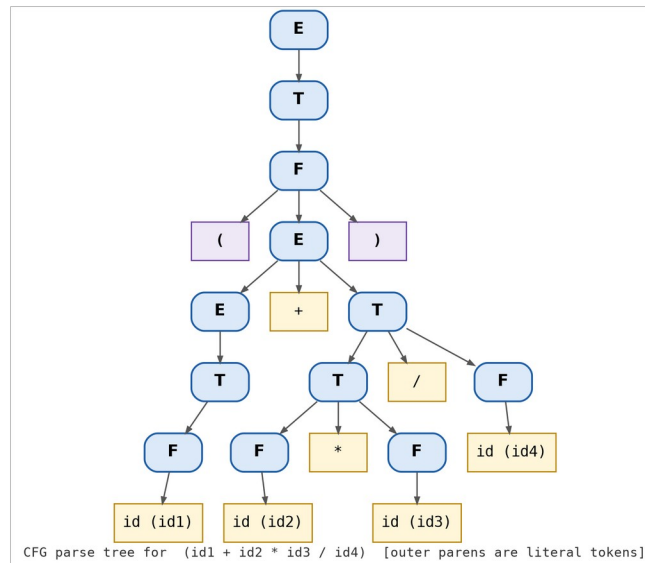
2.1 Parse Tree for $(id + id * id / id)$

Q: Draw the parse tree of nodes for the input sentence $(id + id * id / id)$ using the following attribute grammar. Note that it is important that the ordering of nodes matches the SDD in your drawing.

```
E → E1 + T    { E.node = new Node();  childList = new List();
                childList.add(T.node);
                childList.add(new Leaf('+'));
                childList.add(E1.node);
                E.node.children = childList; }
E → T          { E.node = T.node; }
T → T1 * F     { T.node = new Node();  childList = new List();
                childList.add(F.node);
                childList.add(new Leaf('*'));
                childList.add(T1.node);
                T.node.children = childList; }
T → T1 / F     { T.node = new Node();  childList = new List();
                childList.add(T1.node);
                childList.add(new Leaf('/'));
                childList.add(F.node);
                T.node.children = childList; }
T → F          { T.node = new Node();  childList = new List();
                childList.add(F.node);
                T.node.children = childList; }
F → id         { F.node = new Leaf(id.lexeme); }
F → ( E )     { F.node = E.node; }
```

The input sentence is written WITH its enclosing parentheses — $(id1 + id2 * id3 / id4)$ — so the outer '(' ')' are real tokens that invoke $F \rightarrow (E)$ at the very top, exactly like every other parenthesis in this grammar.

First, the underlying CFG parse tree (this just shows how the grammar derives the string — no semantic actions yet):

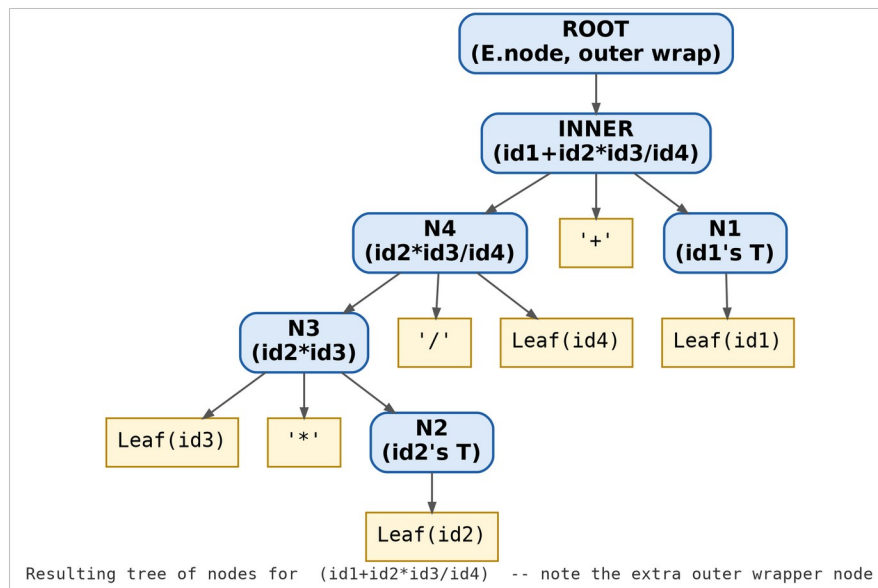


Label the operands id1 + id2 * id3 / id4 in left-to-right input order. Inner structure: $E \rightarrow E1 + T$, where $E1 = id1$, and $T = id2 * id3 / id4$ ($T \rightarrow T1 / F$, $T1 = id2 * id3$, $F = id4$). The whole thing then goes through one more level: $E(\text{outer}) \rightarrow T(\text{outer}) \rightarrow F(\text{outer}) \rightarrow (E(\text{inner}))$.

Step-by-step node construction:

- $F(id1).node = \text{Leaf}(id1)$
- $T(id1).node [T \rightarrow F] = \text{Node}[\text{Leaf}(id1)]$ — call this N1
- $E1.node [E \rightarrow T] = N1$ (no new node)
- $F(id2).node = \text{Leaf}(id2)$; $T(id2).node [T \rightarrow F] = \text{Node}[\text{Leaf}(id2)]$ — call this N2
- $F(id3).node = \text{Leaf}(id3)$
- $T1 * F$ for $id2 * id3$ [$T1 = N2$, $F = \text{Leaf}(id3)$] = $\text{Node}[\text{Leaf}(id3), '*', N2]$ — call this N3
- $F(id4).node = \text{Leaf}(id4)$
- $T1 / F$ for $id2 * id3 / id4$ [$T1 = N3$, $F = \text{Leaf}(id4)$] = $\text{Node}[N3, '/', \text{Leaf}(id4)]$ — call this N4
- Inner $E \rightarrow E1 + T$ [$E1 = N1$, $T = N4$] = $\text{Node}[N4, '+', N1]$ — call this INNER (this is $E(\text{inner}).node$)
- $F(\text{outer}) \rightarrow (E)$: $F.node = \text{INNER}$ (no new node, direct copy of the inner E's value)
- $T(\text{outer}) \rightarrow F$: $T.node = \text{new Node}()$; $\text{childList.add}(F.node = \text{INNER})$; $T.node.children = [\text{INNER}]$ — a NEW wrapper node with exactly ONE child, since $T \rightarrow F$ always wraps in this grammar. Call this ROOT.
- $E(\text{outer}) \rightarrow T$: $E.node = \text{ROOT}$ (no new node) — this is the final answer

Resulting tree diagram (note the extra outer wrapper node caused by the enclosing parentheses):



2.2 Parse Tree for (id / id + id * id)

Q: Draw the parse tree of nodes for the input sentence (id / id + id * id) using the following attribute grammar. Note that it is important that the ordering of nodes matches the SDD in your drawing.

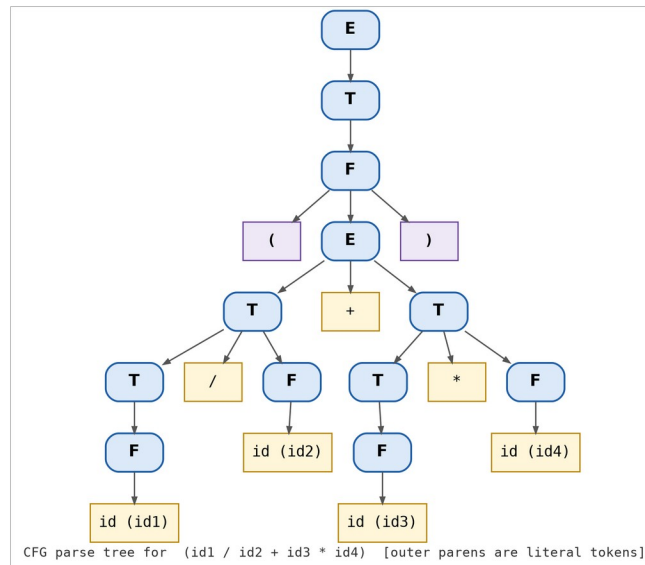
```

E → E1 + T    { E.node = new Node();  childList = new List();
                childList.add(T.node);
                childList.add(new Leaf('+'));
                childList.add(E1.node);
                E.node.children = childList; }
E → T          { E.node = T.node; }
T → T1 * F     { T.node = new Node();  childList = new List();
                childList.add(F.node);
                childList.add(new Leaf('*'));
                childList.add(T1.node);
                T.node.children = childList; }
T → T1 / F     { T.node = new Node();  childList = new List();
                childList.add(T1.node);
                childList.add(new Leaf('/'));
                childList.add(F.node);
                T.node.children = childList; }
T → F          { T.node = new Node();  childList = new List();
                childList.add(F.node);
                T.node.children = childList; }
F → id         { F.node = new Leaf(id.lexeme); }
F → ( E )      { F.node = E.node; }

```

As in 2.1, the input sentence is written WITH its enclosing parentheses — (id1 / id2 + id3 * id4) — so the outer '(' ')' are real tokens that invoke $F \rightarrow (E)$ at the very top.

First, the underlying CFG parse tree (this just shows how the grammar derives the string — no semantic actions yet):

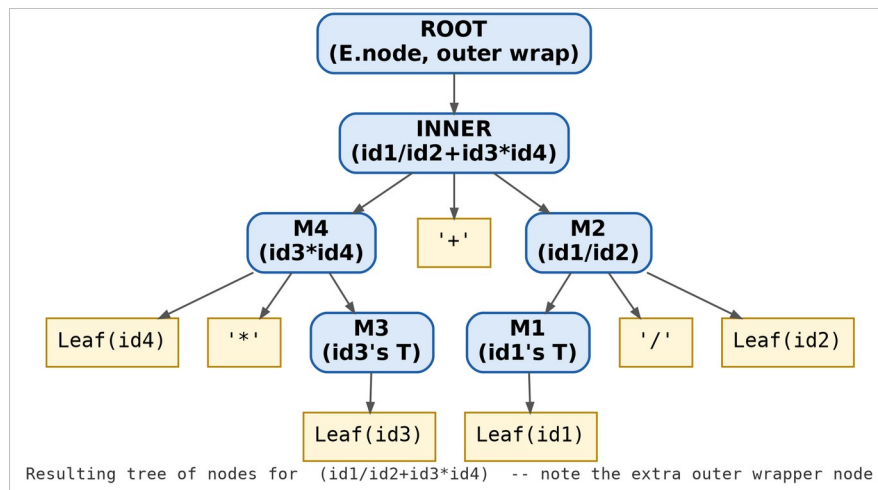


Label the operands id1/id2 + id3*id4. Inner structure: $E \rightarrow E_1 + T$, $E_1 = \text{id1/id2}$ ($T \rightarrow T_1/F$), $T = \text{id3*id4}$ ($T \rightarrow T_1 * F$). The whole thing then goes through one more level: $E(\text{outer}) \rightarrow T(\text{outer}) \rightarrow F(\text{outer}) \rightarrow (E(\text{inner}))$.

Step-by-step node construction:

- $F(\text{id1}).\text{node} = \text{Leaf}(\text{id1})$; $T(\text{id1}).\text{node} [T \rightarrow F] = \text{Node}[\text{Leaf}(\text{id1})]$ — call this M1
- $F(\text{id2}).\text{node} = \text{Leaf}(\text{id2})$
- T_1/F for id1/id2 [$T_1 = M1$, $F = \text{Leaf}(\text{id2})$] = $\text{Node}[M1, '/', \text{Leaf}(\text{id2})]$ — call this M2
- $E_1.\text{node} [E \rightarrow T] = M2$ (no new node)
- $F(\text{id3}).\text{node} = \text{Leaf}(\text{id3})$; $T(\text{id3}).\text{node} [T \rightarrow F] = \text{Node}[\text{Leaf}(\text{id3})]$ — call this M3
- $F(\text{id4}).\text{node} = \text{Leaf}(\text{id4})$
- $T_1 * F$ for id3*id4 [$T_1 = M3$, $F = \text{Leaf}(\text{id4})$] = $\text{Node}[\text{Leaf}(\text{id4}), '*', M3]$ — call this M4
- Inner $E \rightarrow E_1 + T$ [$E_1 = M2$, $T = M4$] = $\text{Node}[M4, '+', M2]$ — call this INNER (this is $E(\text{inner}).\text{node}$)
- $F(\text{outer}) \rightarrow (E)$: $F.\text{node} = \text{INNER}$ (no new node, direct copy of the inner E's value)
- $T(\text{outer}) \rightarrow F$: $T.\text{node} = \text{new Node}()$; $\text{childList.add}(F.\text{node} = \text{INNER})$; $T.\text{node.children} = [\text{INNER}]$ — a NEW wrapper node with exactly ONE child. Call this ROOT.
- $E(\text{outer}) \rightarrow T$: $E.\text{node} = \text{ROOT}$ (no new node) — this is the final answer

Resulting tree diagram (note the extra outer wrapper node caused by the enclosing parentheses):



Theory Questions

Q: Why does $F \rightarrow (E)$ not create a new tree node?

Ans: Parentheses only exist to control grouping/precedence during parsing. Once parsing is complete, that grouping is already reflected in the tree's shape, so the parenthesised expression's node (E.node) is simply reused as F.node — no extra node is needed and the '(' ')' tokens never appear in the tree.

Q: Why does $T \rightarrow F$ still create a new Node instead of just doing $T.node = F.node$?

Ans: This is a design choice specific to this particular grammar — it explicitly wraps a lone factor in its own Node object (with one child), unlike the classic textbook syntax-tree SDD (Example 5.11) where $T \rightarrow F$ simply copies the pointer. You must follow whichever rule the question gives, since it changes the resulting tree shape.

Q: What is the general procedure to simulate such a grammar by hand?

Ans: 1) Parse the input string using the grammar's precedence/associativity to get the derivation structure. 2) Process bottom-up (leaves first): apply the $F \rightarrow id$ and $F \rightarrow (E)$ rules. 3) Apply the T and E rules in the order the parse requires, building each new Node with its children added in exactly the order stated in the rule (not necessarily left-to-right). 4) The final E.node at the top of the derivation is the root of the parse tree of nodes.

3. AST Construction with a Reduce-Count Attribute

This question type asks you to design SDD rules that store, at each AST node, the number of grammar-rule REDUCTIONS used to build that node — i.e., to simulate what an LR parser's reduce steps would look like, counted per subtree.

3.1 General Rule

Grammar used: $E \rightarrow E1 + T \mid T$, $T \rightarrow T1 * F \mid T1 / F \mid F$, $F \rightarrow id \mid (E)$. This SDD builds the actual tree AND stores a count (number of reductions) on every node at the same time — each node object carries both its children and its own .count field.

```

E → E1 + T    { E.node = new Node();
                E.node.children(E1.node);
                E.node.children(new LeafNode('+'));
                E.node.children(T.node);
  
```

```

E → T      { E.node.count = E1.node.count + T.node.count + 1; }
            { E.node = T.node;
              E.node.count = T.node.count + 1; }
T → T1 * F  { T.node = new Node();
              T.node.children(T1.node);
              T.node.children(new LeafNode('*'));
              T.node.children(F.node);
              T.node.count = T1.node.count + F.node.count + 1; }
T → T1 / F  { T.node = new Node();
              T.node.children(T1.node);
              T.node.children(new LeafNode('/'));
              T.node.children(F.node);
              T.node.count = T1.node.count + F.node.count + 1; }
T → F      { T.node = F.node;
              T.node.count = F.node.count + 1; }
F → id     { F.node = new LeafNode(id.lexeme);
              F.node.count = 1; }
F → ( E )  { F.node = E.node;
              F.node.count = E.node.count + 1; }

```

How to apply it (important for simulation):

Every combining rule ($E \rightarrow E1 + T$, $T \rightarrow T1 * F$, $T \rightarrow T1 / F$) adds children in NATURAL left-to-right order (left operand, operator, right operand) and sums the two operand counts plus 1 for its own reduction. The three chain rules ($E \rightarrow T$, $T \rightarrow F$, $F \rightarrow (E)$) do NOT create a new node — they copy the SAME node reference across ($E.node = T.node$, etc.) and simply increment that same node's .count field by 1 in place. A production body always demands a SPECIFIC nonterminal type at each position — $E1$ must be an E , $T1$ must be a T , F must be an F — so each operand's count is obtained by chaining productions UP exactly until it reaches the type the parent production requires:

- An id/subexpression used at an F position stops at $F.count$ (chain ends at $F \rightarrow id$).
- An id/subexpression used at a $T1$ (or lone T) position must reach $T.count$ (one more step, $T \rightarrow F$, if it was only at F level).
- An id/subexpression used at an $E1$ position (the left operand of $+$) must reach $E.count$ (one more step, $E \rightarrow T$, if it was only at T level).

3.2 Reduce Count for $(p + q) * (r / s + t)$

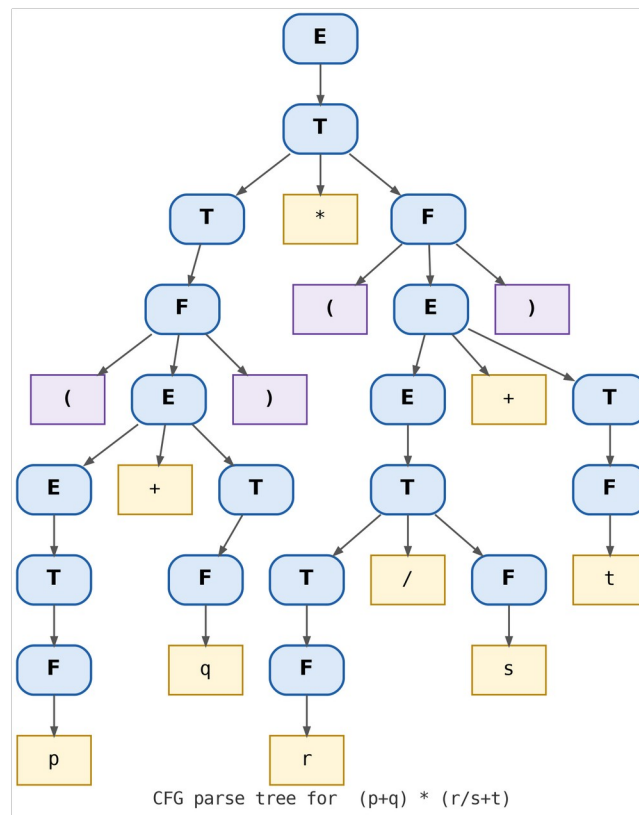
Q: Write the SDD rules for an abstract syntax tree construction for the following grammar that will store a count for the number of reduce operations being done to reach up to the current node for each node in the parse tree. Input String: $(p+q)*(r/s+t)$

```

E → E1 + T  { Write your rules here }
E → T      { Write your rules here }
T → T1 * F  { Write your rules here }
T → T1 / F  { Write your rules here }
T → F      { Write your rules here }
F → id     { Write your rules here }
F → ( E )  { Write your rules here }

```

First, the underlying CFG parse tree for $(p+q)*(r/s+t)$:



Hints given in the question:

For p, the reductions are $F \rightarrow id$, $T \rightarrow F$, $E \rightarrow T$, hence count = 3. For the division node in r/s : r undergoes $F \rightarrow id$ and $T \rightarrow F$ giving 2 reductions, s undergoes $F \rightarrow id$ giving 1 reduction, then $T \rightarrow T/F$ adds 1 reduction, and $E \rightarrow T$ adds 1 reduction, hence count = 2 + 1 + 1 + 1 = 5.

Answer - filled-in rules:

```

E → E1 + T    { E.node = new Node();
                E.node.children(E1.node);
                E.node.children(new LeafNode('+'));
                E.node.children(T.node);
                E.node.count = E1.node.count + T.node.count + 1; }

E → T          { E.node = T.node;
                E.node.count = T.node.count + 1; }

T → T1 * F     { T.node = new Node();
                T.node.children(T1.node);
                T.node.children(new LeafNode('*'));
                T.node.children(F.node);
                T.node.count = T1.node.count + F.node.count + 1; }

T → T1 / F     { T.node = new Node();
                T.node.children(T1.node);
                T.node.children(new LeafNode('/'));
                T.node.children(F.node);
                T.node.count = T1.node.count + F.node.count + 1; }

T → F          { T.node = F.node;
                T.node.count = F.node.count + 1; }

F → id         { F.node = new LeafNode(id.lexeme);
                F.node.count = 1; }

```

```

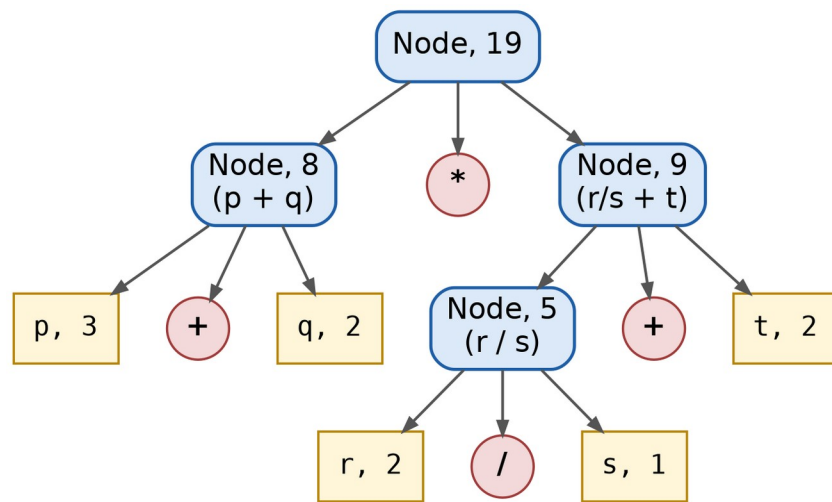
F → ( E )    { F.node = E.node;
               F.node.count = E.node.count + 1; }

```

Full worked simulation:

Sub-expression	Chain of reductions	count
p (as E1 of p+q)	F→id, T→F, E→T	3
q (as T of p+q)	F→id, T→F	2
p+q (as T1 of outer *, via (E)→F→T)	E→E1+T (3+2+1=6), F→(E) (=7), T→F (=8)	8
r (as T1 of r/s)	F→id, T→F	2
s (as F of r/s)	F→id	1
r/s (as E1 of r/s+t, needs E level)	T→T1/F (2+1+1=4), E→T (=5)	5
t (as T of r/s+t)	F→id, T→F	2
r/s+t (as F of outer *, via (E))	E→E1+T (5+2+1=8), F→(E) (=9)	9
Root: (p+q) * (r/s+t)	T→T1*F (8+9+1=18), E→T (=19)	19

Final annotated tree (matches the reference figure):



3.3 Reduce Count for (a*b+c) / (d+e)

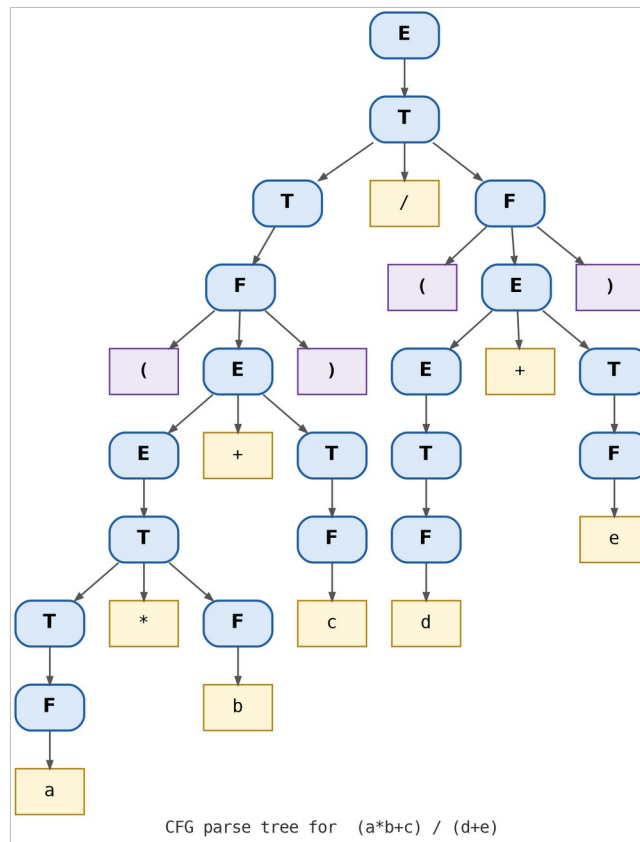
Q: Write the SDD rules for an abstract syntax tree construction for the following grammar that will store a count for the number of reduce operations being done to reach up to the current node for each node in the parse tree. Input String: (a*b+c)/(d+e)

```

E → E1 + T    { Write your rules here }
E → T         { Write your rules here }
T → T1 * F     { Write your rules here }
T → T1 / F     { Write your rules here }
T → F         { Write your rules here }
F → id        { Write your rules here }
F → (E)       { Write your rules here }

```

First, the underlying CFG parse tree for $(a*b+c)/(d+e)$:



Hints given in the question:

For d, the reductions are $F \rightarrow id$, $T \rightarrow F$, $E \rightarrow T$, hence count = 3. For the node corresponding to $a * b$: a undergoes $F \rightarrow id$ and $T \rightarrow F$ giving 2 reductions, b undergoes $F \rightarrow id$ giving 1 reduction, then $T \rightarrow T * F$ adds 1 reduction, and $E \rightarrow T$ adds 1 reduction, hence count = $2 + 1 + 1 + 1 = 5$.

Answer - filled-in rules:

```

E → E1 + T    { E.node = new Node();
                E.node.children(E1.node);
                E.node.children(new LeafNode('+'));
                E.node.children(T.node);
                E.node.count = E1.node.count + T.node.count + 1; }

E → T          { E.node = T.node;
                E.node.count = T.node.count + 1; }

T → T1 * F     { T.node = new Node();
                T.node.children(T1.node);
                T.node.children(new LeafNode('*'));
                T.node.children(F.node);
                T.node.count = T1.node.count + F.node.count + 1; }

T → T1 / F     { T.node = new Node();
                T.node.children(T1.node);
                T.node.children(new LeafNode('/'));
                T.node.children(F.node);
                T.node.count = T1.node.count + F.node.count + 1; }

T → F          { T.node = F.node;
                T.node.count = F.node.count + 1; }

F → id         { F.node = new LeafNode(id.lexeme);

```

```

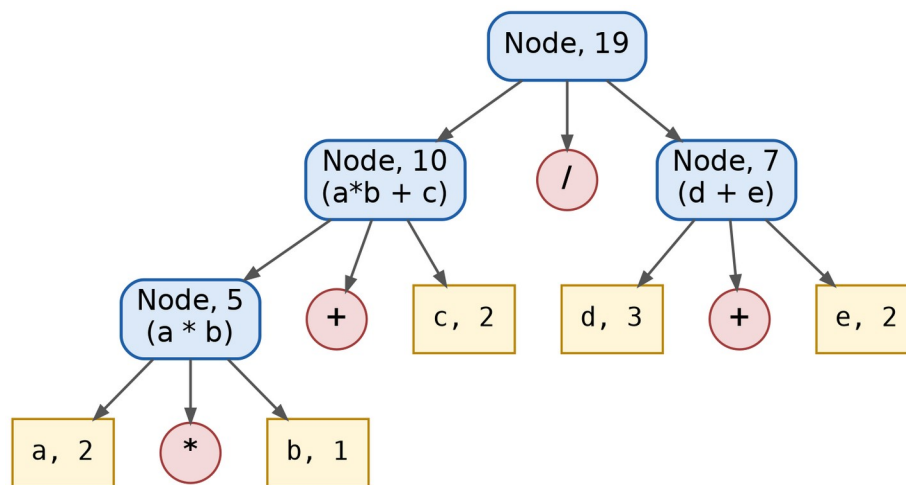
        F.node.count = 1; }
F → ( E ) { F.node = E.node;
           F.node.count = E.node.count + 1; }

```

Full worked simulation:

Sub-expression	Chain of reductions	count
a (as T1 of a*b)	F→id, T→F	2
b (as F of a*b)	F→id	1
a*b (as E1 of a*b+c, needs E level)	T→T1*F (2+1+1=4), E→T (=5)	5
c (as T of a*b+c)	F→id, T→F	2
a*b+c (as T1 of outer /, via (E)→F→T)	E→E1+T (5+2+1=8), F→(E) (=9), T→F (=10)	10
d (as E1 of d+e)	F→id, T→F, E→T	3
e (as T of d+e)	F→id, T→F	2
d+e (as F of outer /, via (E))	E→E1+T (3+2+1=6), F→(E) (=7)	7
Root: (a*b+c) / (d+e)	T→T1/F (10+7+1=18), E→T (=19)	19

Final annotated tree (matches the reference figure):



Theory Questions

Q: Why is the count attribute synthesized and not inherited?

Ans: Because at every node its value is computed purely from the count values of its own children plus 1 (for the reduction that builds this node) — this is exactly the definition of a synthesized attribute: computed bottom-up from a node's children.

Q: Why do count values for the same identifier differ depending on its position (e.g. p=3 but q=2 in the same p+q)?

Ans: Because the grammar requires the LEFT operand of + to be a full E (needing an extra E→T reduction), while the RIGHT operand only needs to be a T. The required nonterminal type at each grammar position — not just the token itself — determines how many chain reductions are needed.

Q: What does the count attribute conceptually represent in terms of LR parsing?

Ans: It represents the number of "reduce" actions an LR parser would perform to turn that portion of the input into the nonterminal required at that point in the derivation — i.e., the size (in reduce-steps) of that subtree's parse history.

4. SDT-1 vs SDT-2 - Placement of Semantic Actions

Q: Consider two SDTs, SDT-1 and SDT-2. In SDT-1 the semantic actions are present only at the end of the production rule. In SDT-2, semantic actions are present at the beginning, in the middle, and at the end of the production body, and there can be multiple semantic actions in a single production rule. Explain which SDT can handle more complex attribute calculation. How would you implement SDT-2?

Ans: SDT-2 can handle more complex attribute calculation. SDT-1 (actions only at the end) is a postfix SDT — it can only compute attributes that are entirely SYNTHESIZED, because by the time the action runs, every child has already been fully processed, so it cannot pass an INHERITED attribute down to a child before that child is parsed. SDT-2, by allowing actions at the start and in the middle of the body, can compute an inherited attribute for a nonterminal and hand it down BEFORE that nonterminal is expanded/parsed — this is required to implement any L-attributed SDD. So SDT-2 is strictly more powerful and can implement the full class of L-attributed translations, not just S-attributed ones.

How to implement SDT-2 (during parsing):

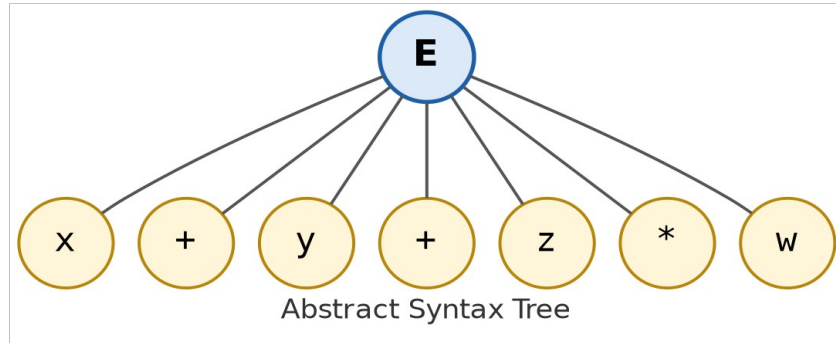
- Replace every embedded action with a marker nonterminal M , where $M \rightarrow \epsilon$, placed exactly at that position in the production body.
- If the resulting grammar (with markers) can still be parsed by the chosen method (top-down/LL is normally required for general L-attributed SDDs) without introducing new parsing conflicts, each marker's ϵ -reduction/expansion point is exactly when its action should fire.
- Follow the two placement rules for L-attributed SDT construction (Section 1.5.4): compute a nonterminal's inherited attributes in an action placed immediately BEFORE that occurrence of the nonterminal; compute the head's synthesized attributes in an action placed at the END of the body.
- If the grammar cannot be parsed with markers inserted (e.g. it creates shift/reduce or predict conflicts), the SDT cannot be executed directly during parsing — instead, build the full parse tree first, then execute the actions during a preorder (left-to-right, depth-first) traversal of the tree.

5. Filling In Missing SDT Rules for Syntax-Tree Construction

This question type gives an incomplete SDT (some rules blanked out) plus the finished abstract syntax tree, and asks you to fill in the missing rule(s) so they are consistent with each other and with the standard syntax-tree-building pattern (see textbook Example 5.11/5.12: `E.node = new Node(op, leftChild.node, rightChild.node)`).

5.1 Input: $x + ((y + z) * w)$

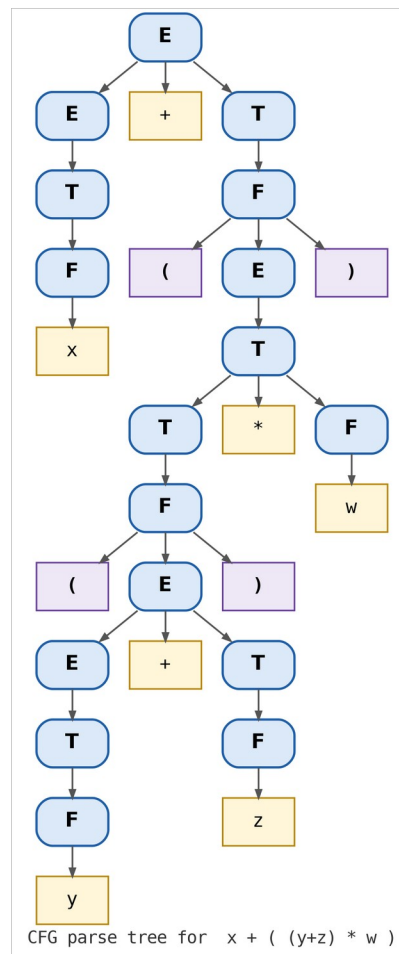
Q: You are provided with an abstract syntax tree and a set of productions of a grammar - for the input sentence $x + (y + z) * w$. You have to fill in the missing rules so that your semantic rules match the ordering of the nodes in the parse tree.



Grammar as given (blank):

```
E → E1 + T {Write your rules here}
E → T      {E.node = T.node;}
T → T1 * F {Write your rules here}
T → F      {T.node = F.node}
F → id     {Write your rules here}
F → (E)    {F.node = E.node}
```

First, the underlying CFG parse tree for $x + ((y+z)*w)$:



Answer - filled-in rules (the accepted exam-solution style, with flattening):

```

E → E1 + T { n = newNode();
              if (E1.node.child != empty) { n.child.append(E1.node.child); }
              else { n.child.append(E1.node); }
              n.child.append("+");
              if (T.node.child != empty) { n.child.append(T.node.child); }
              else { n.child.append(T.node); }
              E.node = n; } <- filled in

E → T { E.node = T.node; }

T → T1 * F { n = newNode();
              if (T1.node.child != empty) { n.child.append(T1.node.child); }
              else { n.child.append(T1.node); }
              n.child.append("*");
              if (F.node.child != empty) { n.child.append(F.node.child); }
              else { n.child.append(F.node); }
              T.node = n; } <- filled in

T → F { T.node = F.node; }

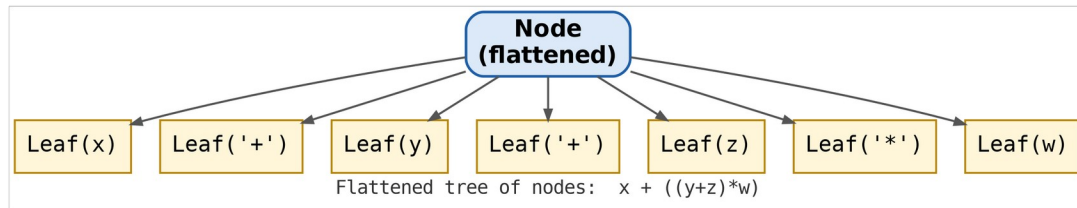
F → id { F.node = new Leaf(id.lexeme); } <- filled in

F → (E) { F.node = E.node; }
  
```

Notice the "if child != empty" check: whenever an operand is already a constructed Node (not a bare Leaf), its children are spliced directly into the new node instead of being nested one level deeper. Applied to $x + ((y+z)*w)$: x , y , z , w start as bare Leaf nodes (no children). $y+z$ combines two bare leaves into $\text{Node}[y, '+', z]$ —

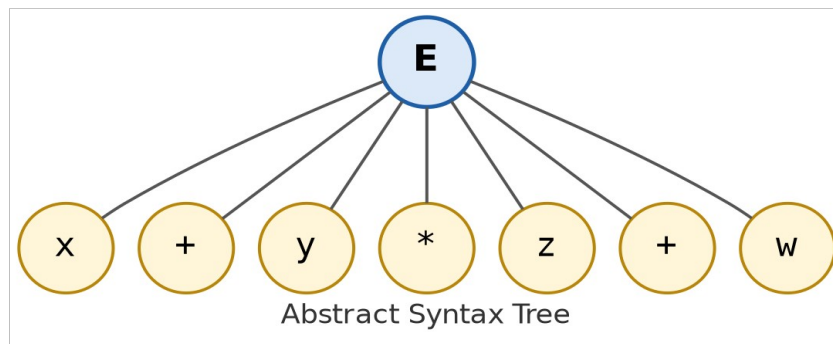
now it HAS children. $(y+z)*w$: T1 = that node (has children, so flatten/splice) gives $[y, '+', z, '*', w]$. Finally $x +$ (...): E1 = Leaf(x) (no children, added as-is) and T = the node above (has children, so flatten/splice) — everything lands as ONE flat node with all 7 tokens as direct children, in their original left-to-right order.

Correctly nested syntax tree (built by applying the filled-in rules):



5.2 Input: $x + ((y * z) + w)$

Q: You are provided with an abstract syntax tree and a set of productions of a grammar – for the input sentence $x + ((y * z) + w)$. You have to fill in the missing rules so that your semantic rules match the ordering of the nodes in the parse tree.



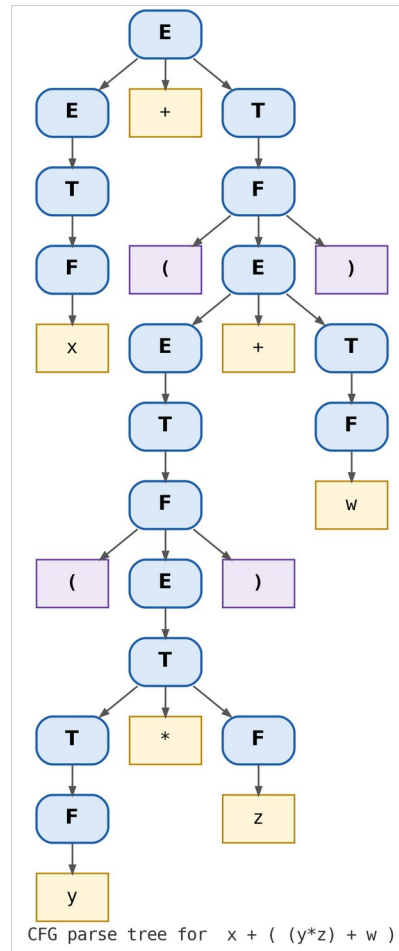
Grammar as given (blank):

```

E → E1 + T {Write your rules here}
E → T      {E.node = T.node;}
T → T1 * F {Write your rules here}
T → F      {T.node = F.node}
F → id     {Write your rules here}
F → (E)    {F.node = E.node}

```

First, the underlying CFG parse tree for $x + ((y*z)+w)$:



Answer - filled-in rules (the accepted exam-solution style, with flattening):

```

E → E1 + T { n = newNode();
              if (E1.node.child != empty) { n.child.append(E1.node.child); }
              else { n.child.append(E1.node); }
              n.child.append("+");
              if (T.node.child != empty) { n.child.append(T.node.child); }
              else { n.child.append(T.node); }
              E.node = n; } <- filled in

E → T { E.node = T.node; }

T → T1 * F { n = newNode();
              if (T1.node.child != empty) { n.child.append(T1.node.child); }
              else { n.child.append(T1.node); }
              n.child.append("*");
              if (F.node.child != empty) { n.child.append(F.node.child); }
              else { n.child.append(F.node); }
              T.node = n; } <- filled in

T → F { T.node = F.node; }

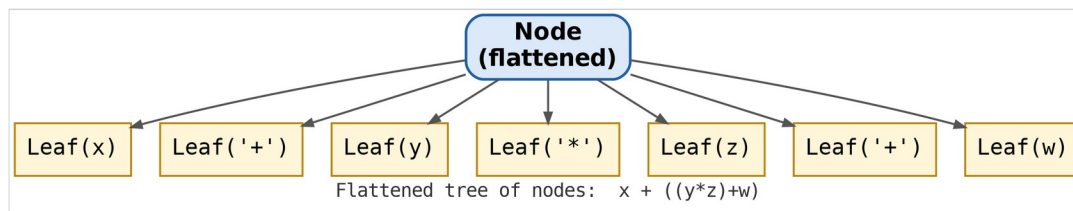
F → id { F.node = new Leaf(id.lexeme); } <- filled in

F → (E) { F.node = E.node; }
  
```

Applied to $x + ((y*z)+w)$: $y*z$ combines two bare leaves into $\text{Node}[y, '*', z]$ — now it HAS children. $(y*z)+w$: $E1 =$ that node (has children, so flatten/splice) gives $[y, '*', z, '+', w]$. Finally $x + (...)$: $E1 = \text{Leaf}(x)$ (no children, added

as-is) and T = the node above (has children, so flatten/splice) — everything lands as ONE flat node with all 7 tokens as direct children, in their original left-to-right order.

Correctly nested syntax tree (built by applying the filled-in rules):



Theory Questions

Q: What is the general pattern for writing a synthesized-attribute rule that builds a syntax-tree node for a binary operator production $A \rightarrow B \text{ op } C$?

Ans: Create one new node n. Check each operand before adding it: if it already has children (i.e. it's a constructed Node, not a bare Leaf), splice its children directly into n's list; otherwise add the operand itself as one item. Add the operator leaf in between, in natural left-to-right order: B first, then 'op', then C. This flattening is exactly the pattern used in the worked answers above, and it's why a chain of operators with no parentheses collapses into one flat node.

Q: What is the rule for a leaf (terminal) production such as $F \rightarrow id$ or $T \rightarrow num$?

Ans: $F.node = \text{new Leaf}(id.lexeme)$ (or $\text{new Leaf}(num.val)$) — create a leaf object holding the terminal's lexical value; it has no children.

Q: Why do chain productions like $E \rightarrow T$ or $F \rightarrow (E)$ not create a new node?

Ans: Because they add no new information to the syntax tree — the nonterminal on the right already has a fully built subtree, so the head simply reuses (copies the pointer to) that same node: $E.node = T.node$, or $F.node = E.node$.

Q: Is the syntax-tree SDD of this type (Example 5.11) S-attributed or L-attributed?

Ans: It is S-attributed — every attribute (node) is synthesized, built bottom-up purely from children, with no inherited attributes at all. This means it can be evaluated naturally during a bottom-up (LR) parse.

6. Practice Sheet

These extra problems are written at the same difficulty level as the previous-semester questions above, in the same exam format, but with different expression structures so you can test your understanding rather than just recall. Try to solve each one yourself first, then check against the full solution (CFG parse tree included) given below it.

6.1 Parse Tree of Nodes

Q: Draw the parse tree of nodes for the input sentence $(id + id) * id / id$ using the following attribute grammar. Note that it is important that the ordering of nodes matches the SDD in your drawing.

```

E → E1 + T    { E.node = new Node();  childList = new List();
                  childList.add(T.node);
                  childList.add(new Leaf('+'));
                  childList.add(E1.node);
                  E.node.children = childList; }
E → T          { E.node = T.node; }

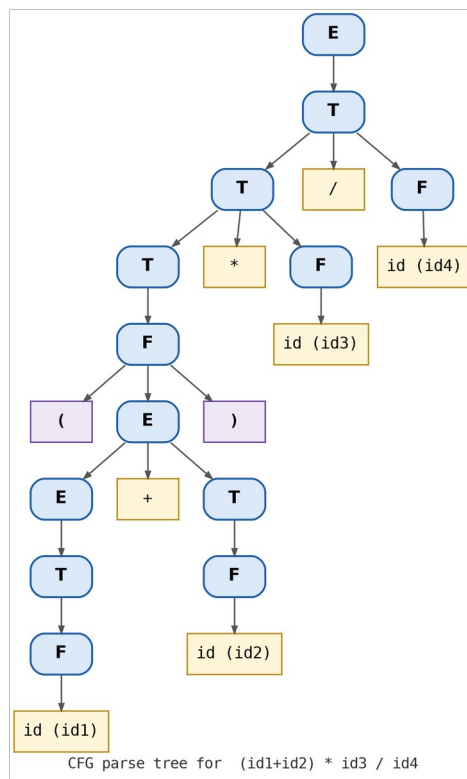
```

```

T → T1 * F    { T.node = new Node();  childList = new List();
                childList.add(F.node);
                childList.add(new Leaf('*'));
                childList.add(T1.node);
                T.node.children = childList; }
T → T1 / F    { T.node = new Node();  childList = new List();
                childList.add(T1.node);
                childList.add(new Leaf('/'));
                childList.add(F.node);
                T.node.children = childList; }
T → F         { T.node = new Node();  childList = new List();
                childList.add(F.node);
                T.node.children = childList; }
F → id        { F.node = new Leaf(id.lexeme); }
F → ( E )    { F.node = E.node; }

```

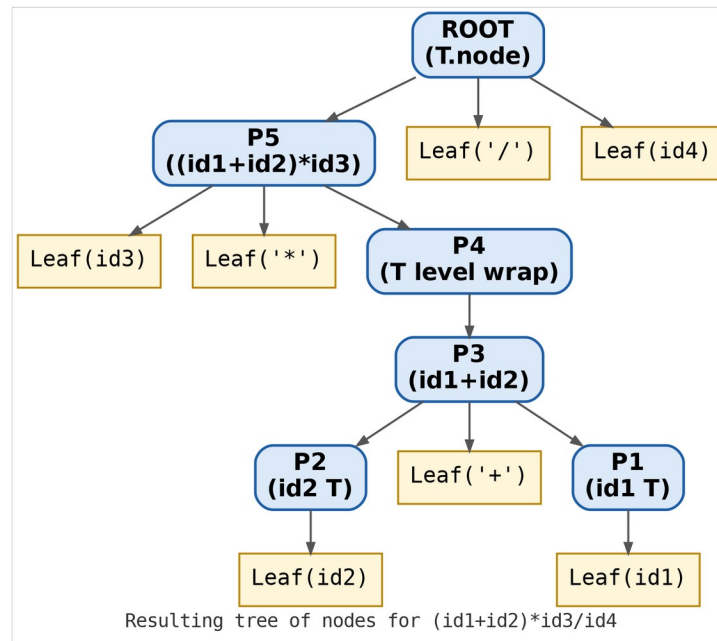
First, the underlying CFG parse tree for (id1+id2) * id3 / id4:



Step-by-step node construction:

- $F(id1).node = \text{Leaf}(id1)$; $T(id1).node [T \rightarrow F] = \text{Node}[\text{Leaf}(id1)]$ — call this P1
- $F(id2).node = \text{Leaf}(id2)$; $T(id2).node [T \rightarrow F] = \text{Node}[\text{Leaf}(id2)]$ — call this P2
- $E(id1+id2) [E \rightarrow E1+T, E1=P1, T=P2] = \text{Node}[P2, '+', P1]$ — call this P3
- $F((id1+id2)).node [F \rightarrow (E)] = P3$ (no new node — parentheses vanish)
- $T1$ for the outer $*$ position $[T \rightarrow F \text{ chain, since } (id1+id2) \text{ needs } T \text{ level}] = \text{Node}[P3]$ — call this P4
- $F(id3).node = \text{Leaf}(id3)$
- $T1 * F$ for $(id1+id2) * id3$ $[T1 = P4, F = \text{Leaf}(id3)] = \text{Node}[\text{Leaf}(id3), '*', P4]$ — call this P5
- $F(id4).node = \text{Leaf}(id4)$
- Root: $T1/F$ for the whole expression $[T1 = P5, F = \text{Leaf}(id4)] = \text{Node}[P5, '/', \text{Leaf}(id4)]$ — the final tree

Resulting tree of nodes:

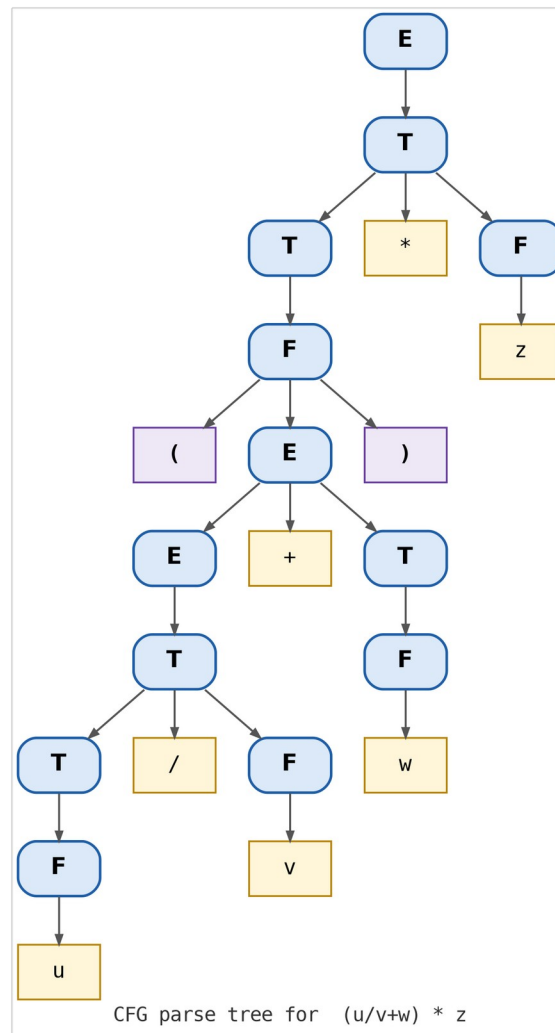


6.2 Reduce-Count SDD

Q: Write the SDD rules for an abstract syntax tree construction for the following grammar that will store a count for the number of reduce operations being done to reach up to the current node for each node in the parse tree. Input String: (u/v+w)*z

$E \rightarrow E1 + T$	{ Write your rules here }
$E \rightarrow T$	{ Write your rules here }
$T \rightarrow T1 * F$	{ Write your rules here }
$T \rightarrow T1 / F$	{ Write your rules here }
$T \rightarrow F$	{ Write your rules here }
$F \rightarrow id$	{ Write your rules here }
$F \rightarrow (E)$	{ Write your rules here }

First, the underlying CFG parse tree for $(u/v+w)*z$:



Hints:

For z , the only reduction is $F \rightarrow id$, hence count = 1 (z is used directly as F on the right of $*$, so no further promotion is needed). For u/v : u undergoes $F \rightarrow id$ and $T \rightarrow F$ giving 2 reductions, v undergoes $F \rightarrow id$ giving 1 reduction, then $T \rightarrow T1/F$ adds 1 reduction, and $E \rightarrow T$ adds 1 more reduction (since u/v is the left operand of the $+$), hence count = $2 + 1 + 1 + 1 = 5$.

Answer - filled-in rules:

```

E → E1 + T    { E.node = new Node();
                E.node.children(E1.node);
                E.node.children(new LeafNode('+'));
                E.node.children(T.node);
                E.node.count = E1.node.count + T.node.count + 1; }
E → T          { E.node = T.node;
                E.node.count = T.node.count + 1; }
T → T1 * F     { T.node = new Node();
                T.node.children(T1.node);
                T.node.children(new LeafNode('*'));
                T.node.children(F.node);
                T.node.count = T1.node.count + F.node.count + 1; }
  
```

```

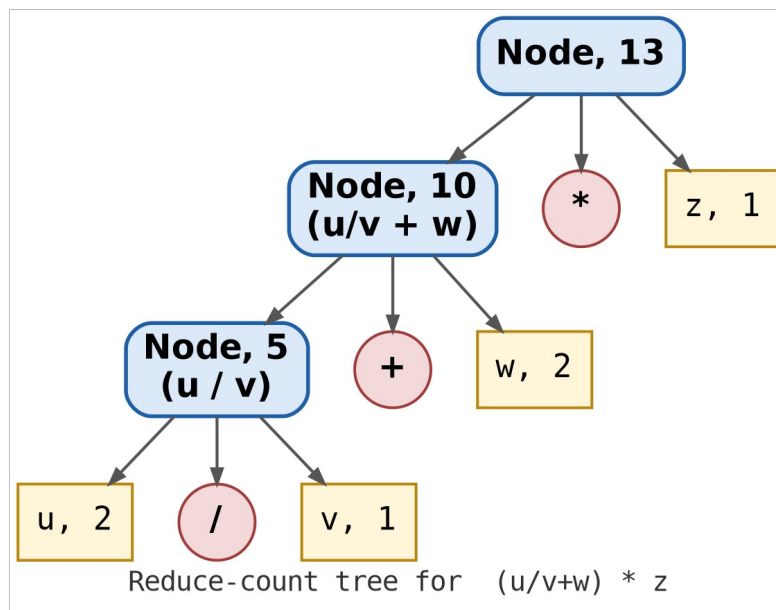
T → T1 / F    { T.node = new Node();
                  T.node.children(T1.node);
                  T.node.children(new LeafNode('/'));
                  T.node.children(F.node);
                  T.node.count = T1.node.count + F.node.count + 1; }
T → F          { T.node = F.node;
                  T.node.count = F.node.count + 1; }
F → id         { F.node = new LeafNode(id.lexeme);
                  F.node.count = 1; }
F → ( E )      { F.node = E.node;
                  F.node.count = E.node.count + 1; }

```

Full worked simulation:

Sub-expression	Chain of reductions	count
u (as T1 of u/v)	F→id, T→F	2
v (as F of u/v)	F→id	1
u/v (as E1 of u/v+w, needs E level)	T→T1/F (2+1+1=4), E→T (=5)	5
w (as T of u/v+w)	F→id, T→F	2
(u/v+w) (as T1 of outer *, via (E)→F→T)	E→E1+T (5+2+1=8), F→(E) (=9), T→F (=10)	10
z (as F of outer *, plain id)	F→id	1
Root: (u/v+w) * z	T→T1*F (10+1+1=12), E→T (=13)	13

Final annotated tree:



6.3 Filling In Missing SDT Rules

This problem uses the same grammar shape as before ($E \rightarrow E1+T \mid T$, $T \rightarrow T1*F \mid F$, $F \rightarrow id \mid (E)$), but this time the rules are filled in using the compact one-node-per-call style — `new Node(op, left.node, right.node)` — instead of the `childList` style. Both are valid ways to build the same kind of syntax tree; you should be comfortable writing either.

Q: You are provided with a set of productions of a grammar – for the input sentence $(p + q) * (r + s)$. You have to fill in the missing rules so that your semantic rules match the ordering of the nodes in the parse tree.

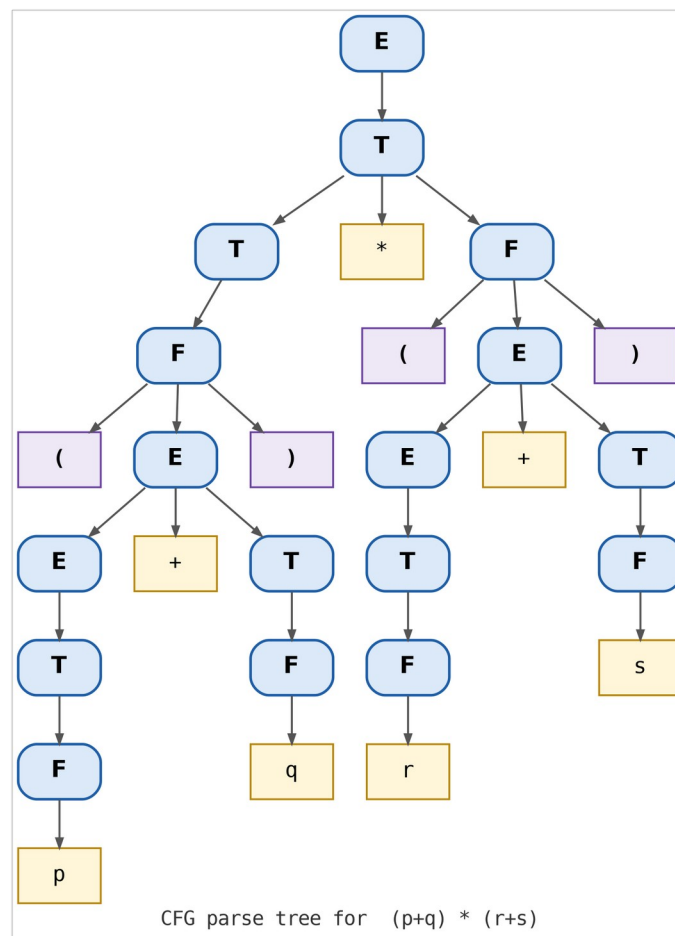
Grammar as given (blank):

```

E → E1 + T {Write your rules here}
E → T      {E.node = T.node;}
T → T1 * F {Write your rules here}
T → F      {T.node = F.node;}
F → id     {Write your rules here}
F → ( E )  {F.node = E.node;}

```

First, the underlying CFG parse tree for $(p+q) * (r+s)$:



Answer - filled-in rules:

```

E → E1 + T { E.node = new Node('+', E1.node, T.node); } <- filled in
E → T      { E.node = T.node; }
T → T1 * F { T.node = new Node('*', T1.node, F.node); } <- filled in
T → F      { T.node = F.node; }
F → id     { F.node = new Leaf(id.lexeme); } <- filled in
F → ( E )  { F.node = E.node; }

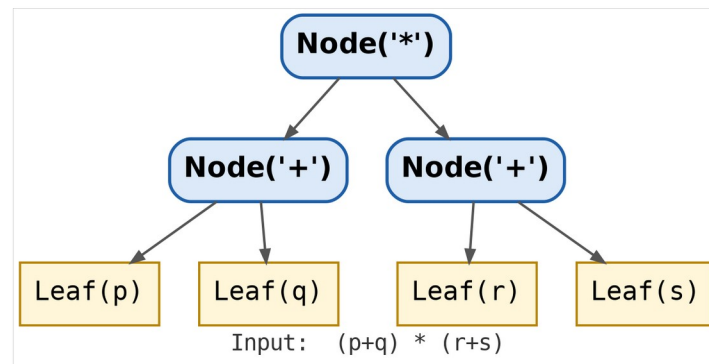
```

Applied to $(p+q) * (r+s)$: $p+q$ becomes $\text{Node}('+', \text{Leaf}(p), \text{Leaf}(q))$ via $E \rightarrow E1 + T$, and this whole thing reaches $T1$ by the chain $F \rightarrow (E)$ then $T \rightarrow F$ (no new node at either chain step, since both are simple pointer copies).

Likewise $r+s$ becomes $\text{Node}('+', \text{Leaf}(r), \text{Leaf}(s))$ and reaches F by $F \rightarrow (E)$. Finally the whole expression

becomes $\text{Node}('*', \text{Node}('+', p, q), \text{Node}('+', r, s))$ via the outer $T \rightarrow T_1 * F$ — notice there is no leading identifier this time, so the root is a '*' node with two '+' node children.

Correctly nested syntax tree (built by applying the filled-in rules):



6.4 Filling In Missing SDT Rules - Field-Based Style

Q: You are provided with a set of productions of a grammar – for the input sentence $(m + n) * o$. This time, instead of building a `childList`, each `Node` object simply has three fields: `op` (the operator), `left` (the left child), and `right` (the right child). Fill in the missing rules so that your semantic rules match the ordering of the nodes in the parse tree.

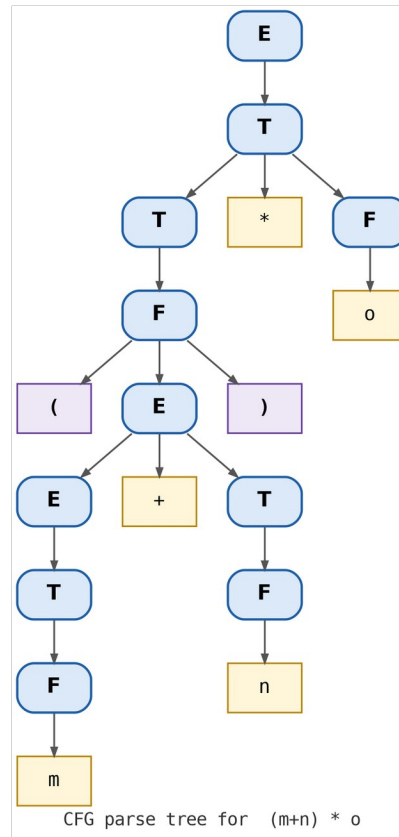
Grammar as given (blank):

```

E → E1 + T {Write your rules here}
E → T      {E.node = T.node;}
T → T1 * F {Write your rules here}
T → F      {T.node = F.node}
F → id     {Write your rules here}
F → (E)    {F.node = E.node}

```

First, the underlying CFG parse tree for $(m+n) * o$:



Answer - filled-in rules (field-based: op / left / right, not a childList):

```

E → E1 + T    { E.node = new Node();
                E.node.op = '+';
                E.node.left = E1.node;
                E.node.right = T.node; }      <- filled in
E → T          { E.node = T.node; }
T → T1 * F     { T.node = new Node();
                T.node.op = '*';
                T.node.left = T1.node;
                T.node.right = F.node; }      <- filled in
T → F          { T.node = F.node; }
F → id         { F.node = new Leaf(id.lexeme); }  <- filled in
F → ( E )      { F.node = E.node; }
  
```

Applied to $(m+n) * o$: $m+n$ becomes a new Node with $op='+'$, $left=Leaf(m)$, $right=Leaf(n)$ via $E \rightarrow E1+T$; this whole Node reaches $T1$ by the chain $F \rightarrow (E)$ then $T \rightarrow F$ (both simple pointer copies, no new node). Finally the whole expression becomes a new Node with $op='*'$, $left=$ that $(m+n)$ Node, $right=Leaf(o)$ via the outer $T \rightarrow T1 * F$. Notice each Node now stores its operator as a field rather than as an entry in a list, so every interior node has exactly 2 children (left, right) instead of 3.

Resulting tree (op stored as a field, not a childList entry):

